

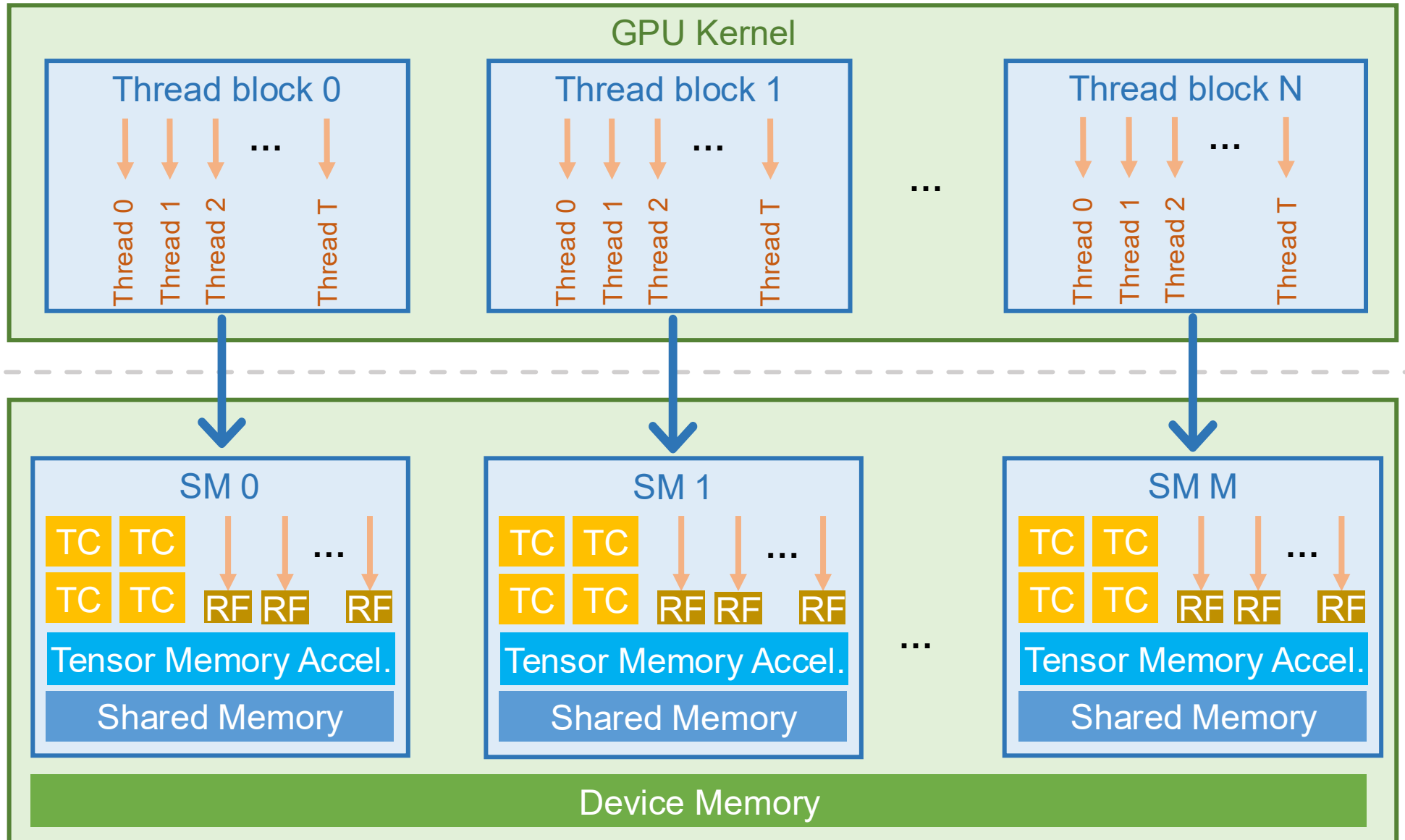
15-442/15-642: Machine Learning Systems

Advanced Topics: Mega-Kernel

Tianqi Chen and Zhihao Jia
Carnegie Mellon University

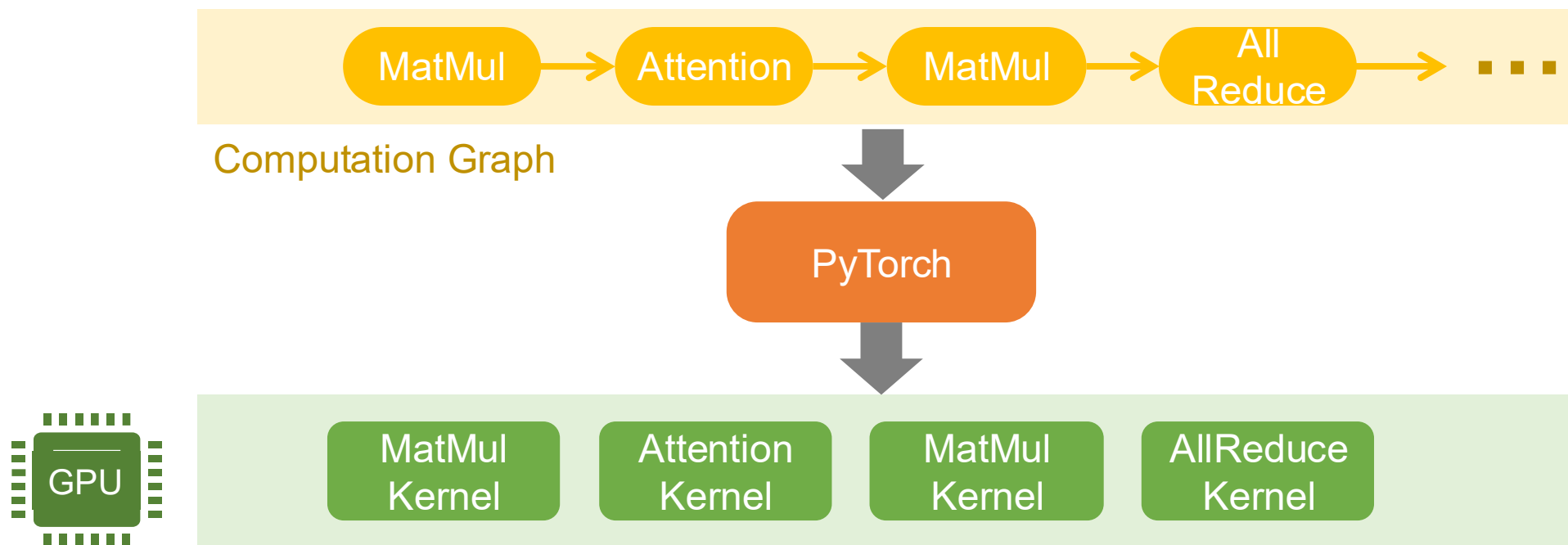
What is a (Mega-)Kernel?

Programming
Abstraction

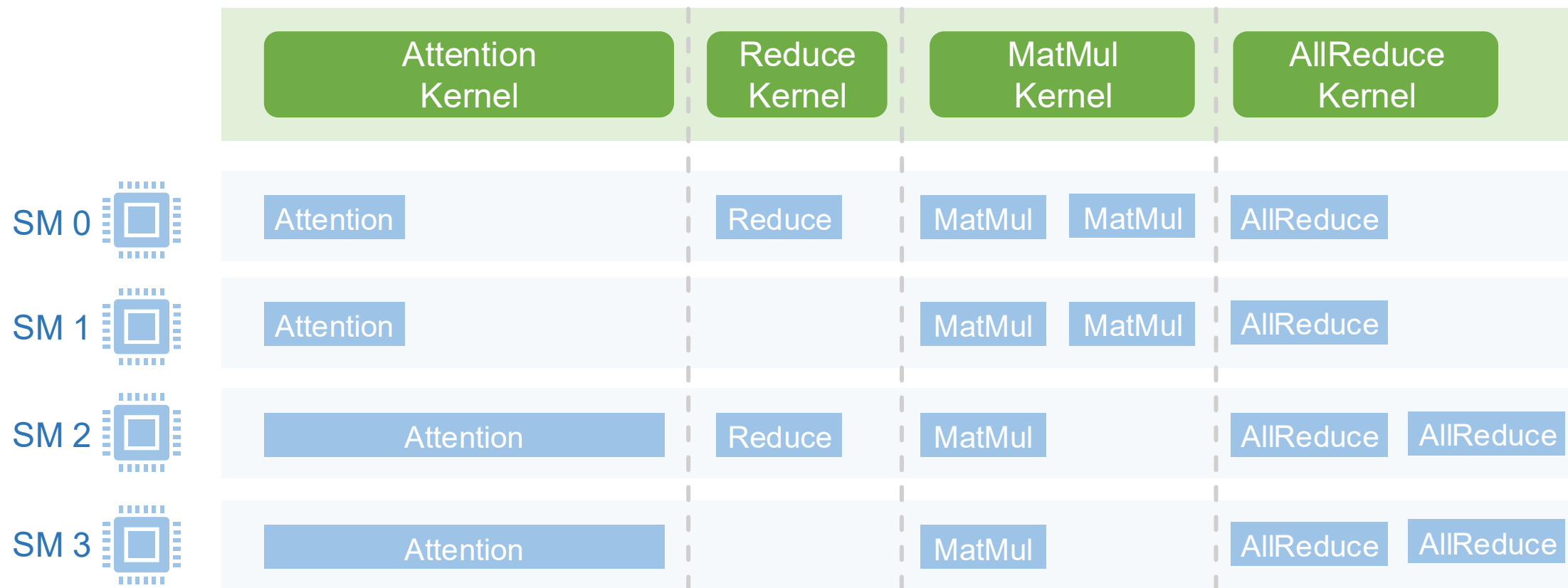


Hardware
Architecture

Existing Kernel-Per-Operator Approach



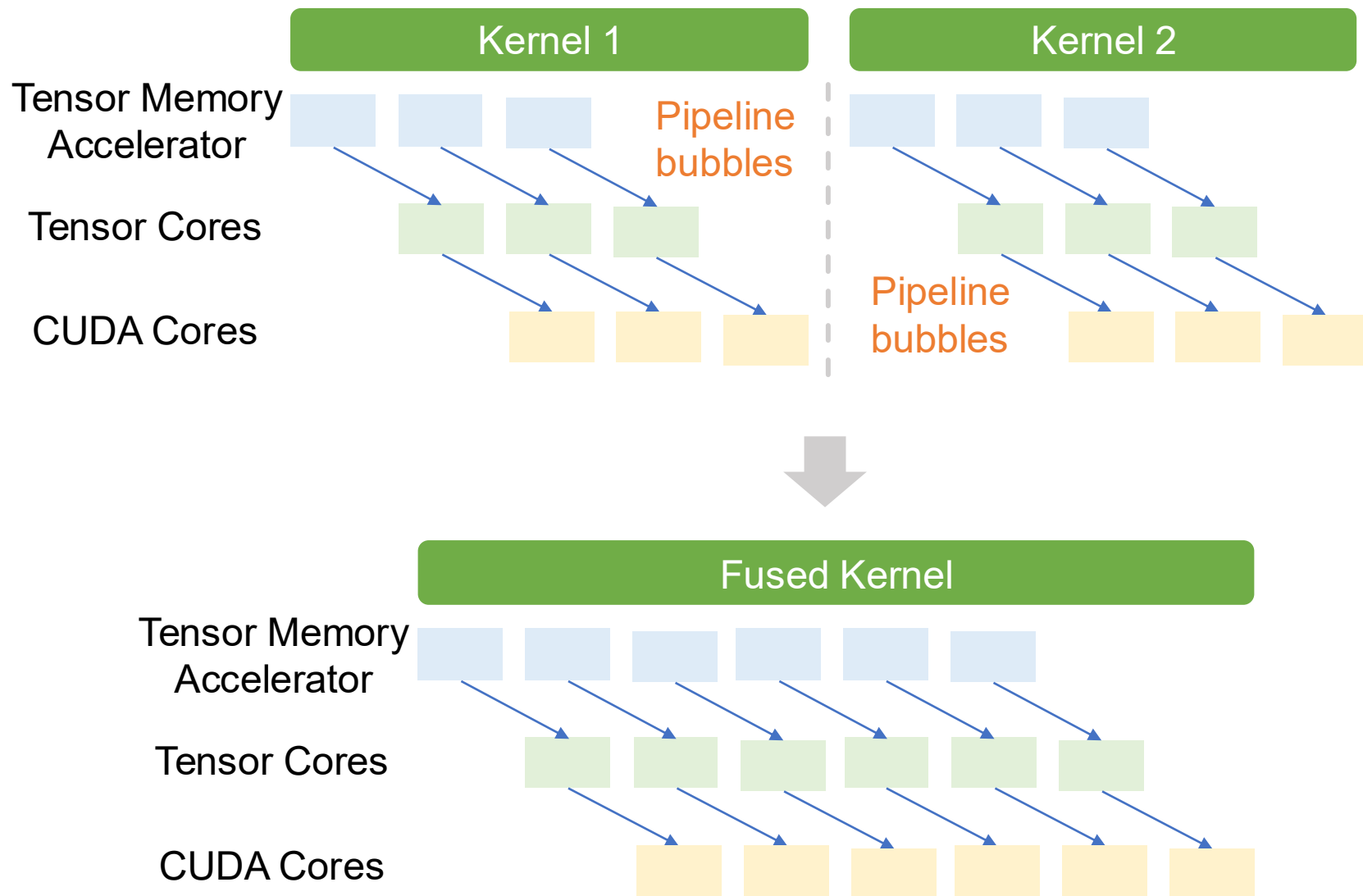
Existing Kernel-Per-Operator Approach



Limitations

No Inter-Layer Pipelining

Kernel barriers prevent inter-layer pipelining



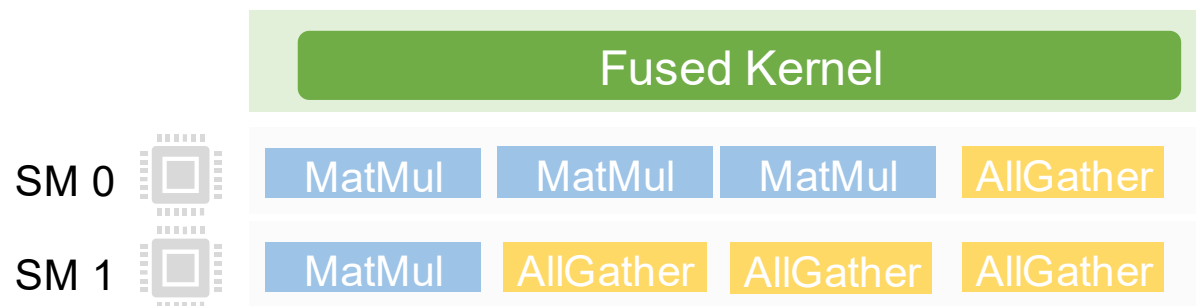
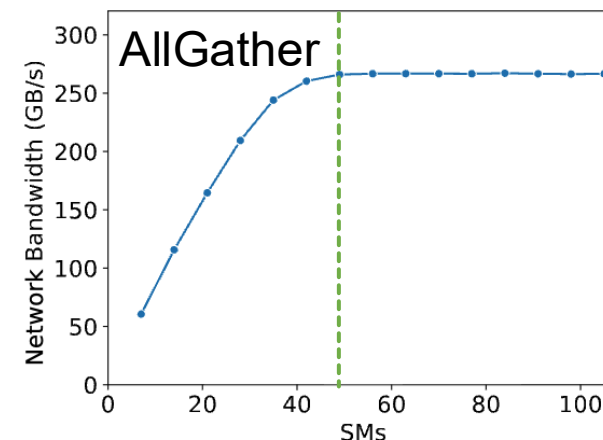
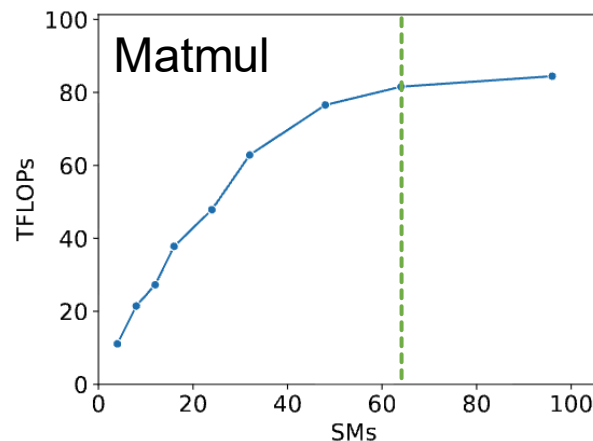
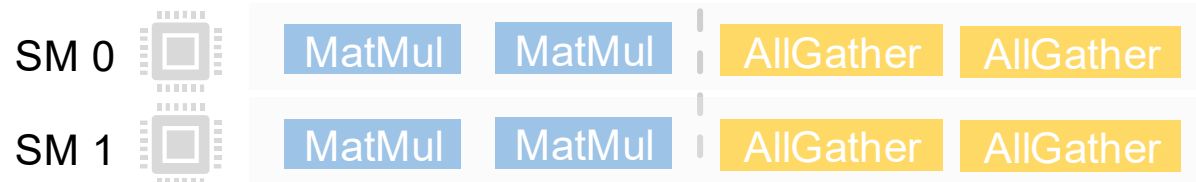
Limitations

No Inter-Layer Pipelining

Kernel barriers prevent inter-layer pipelining

No Overlapping

Coarse-grained dependency prevents comp. & comm. overlap



Limitations

No Inter-Layer Pipelining

Kernel barriers prevent inter-layer pipelining

No Overlapping

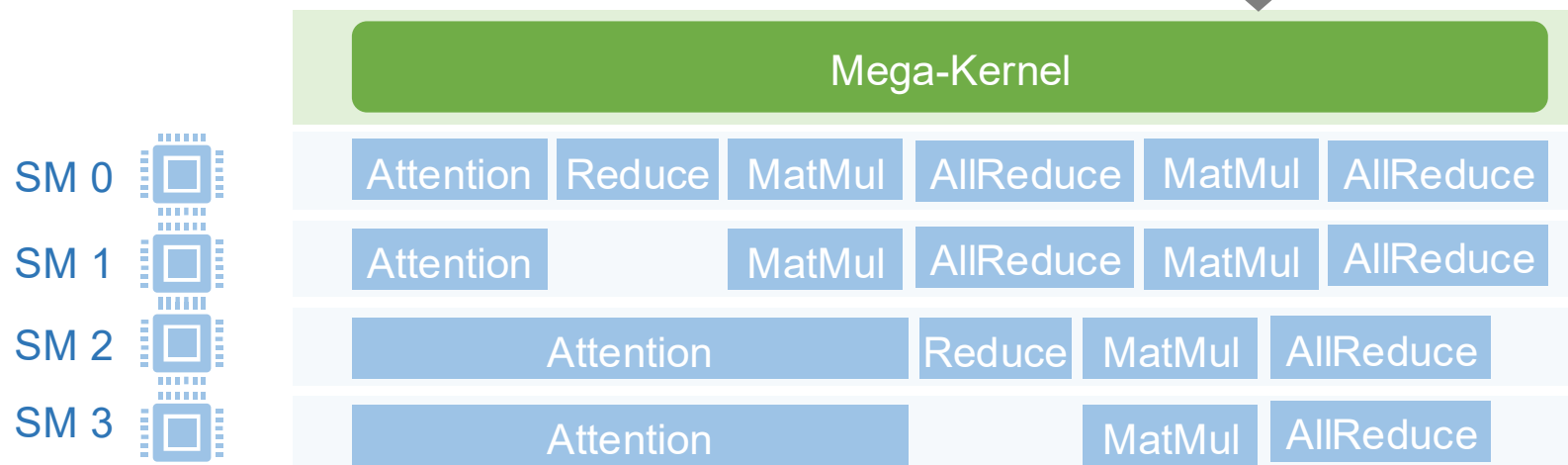
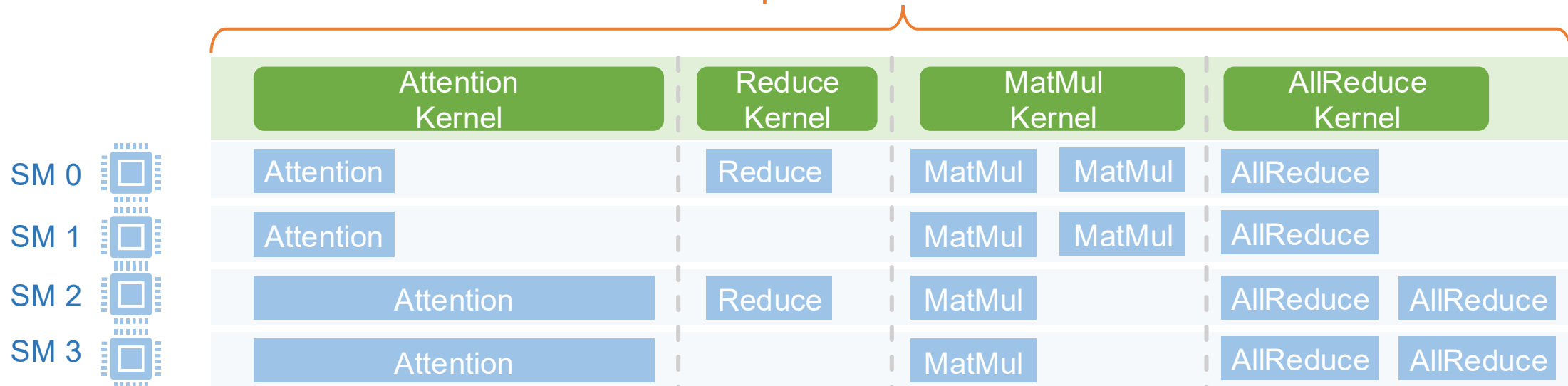
Coarse-grained dependency prevents comp. & comm. overlap

Limited Dynamism

Rely on CUDA graphs to reduce kernel launch overhead

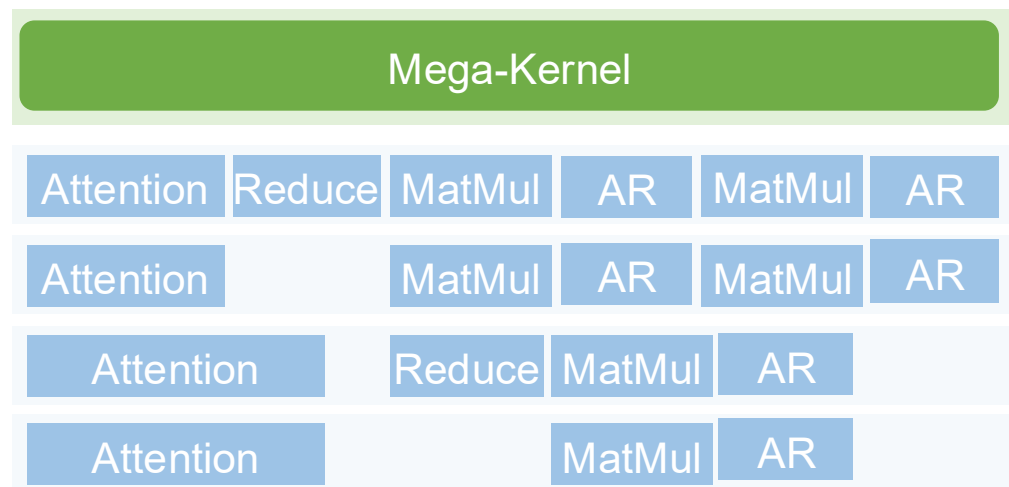
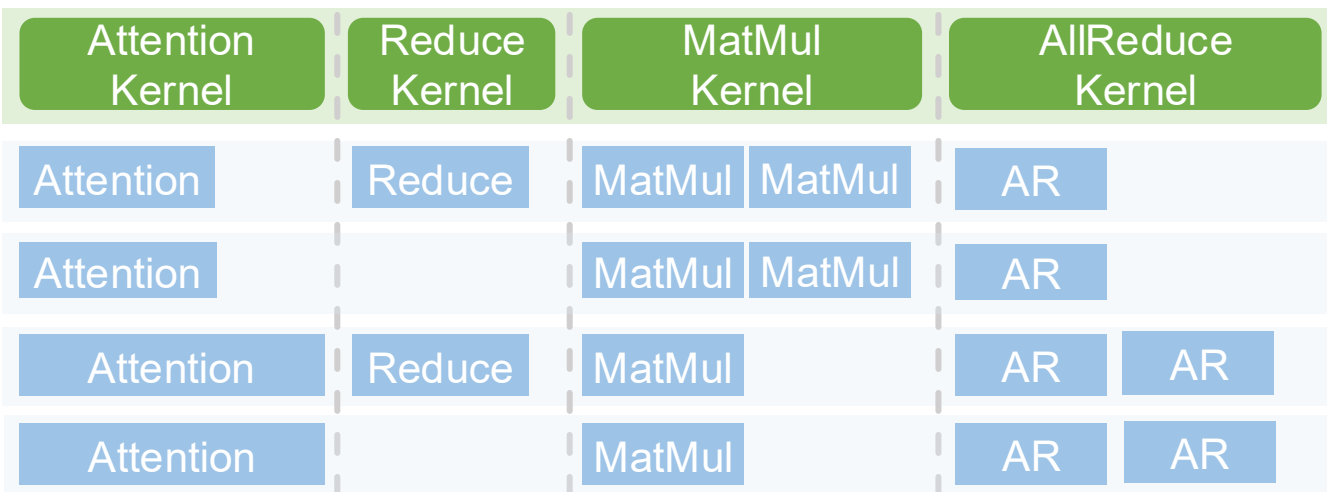
Kernel-Per-Operator v.s. Mega-Kernel

An LLM forward pass launches 100s-1000s kernels



- ✓ No kernel barriers
- ✓ Operator reordering
- ✓ Load balancing

Advantages of Mega-Kernel



Inter-Layer Pipelining

All layers are fused in the same mega-kernel

Overlapping Comp/Comm

Fine-grained dependency + operator reordering

Dynamic Workloads

No need for CUDA graphs + load balancing

Key Challenges

1. How to manage dependency?

No kernel barriers in mega-kernel

Task Graph

2. How to handle dynamism?

Continuous batching, prefill/decode,
paged/radix attention, speculative decoding

In-Kernel
Parallel Runtime

3. How to optimize performance?

Existing compilers target individual kernels

Mirage Superoptimizer*

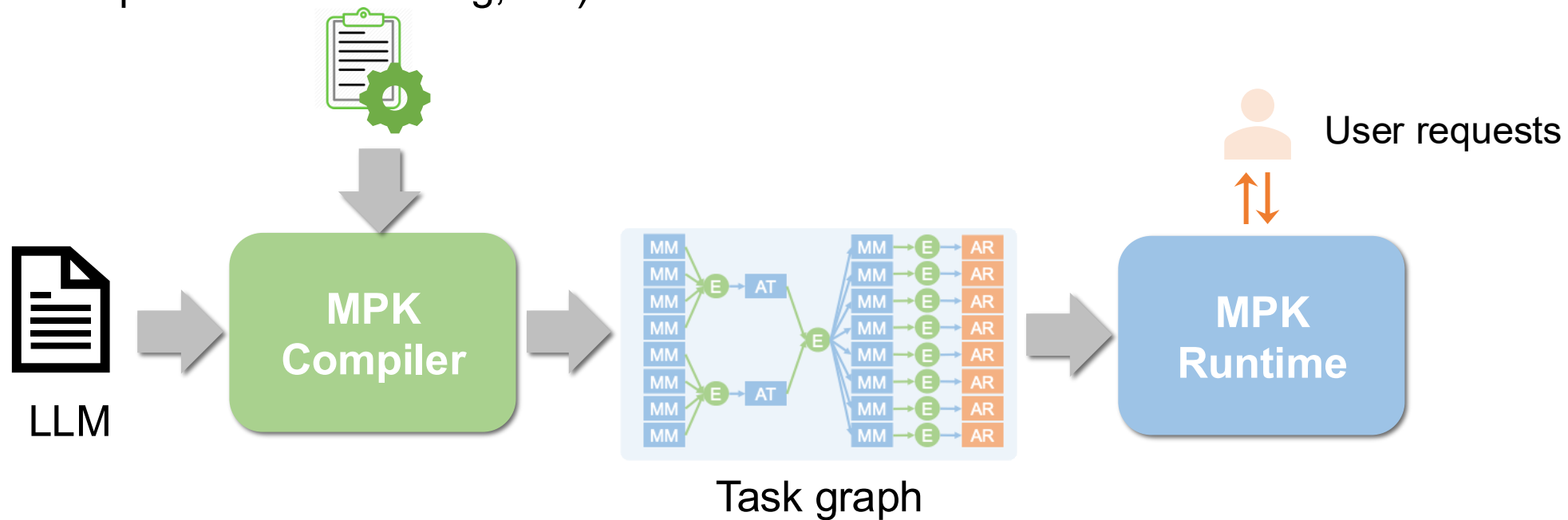
Mirage Persistent Kernel: Compiling LLM Serving into a Mega-Kernel



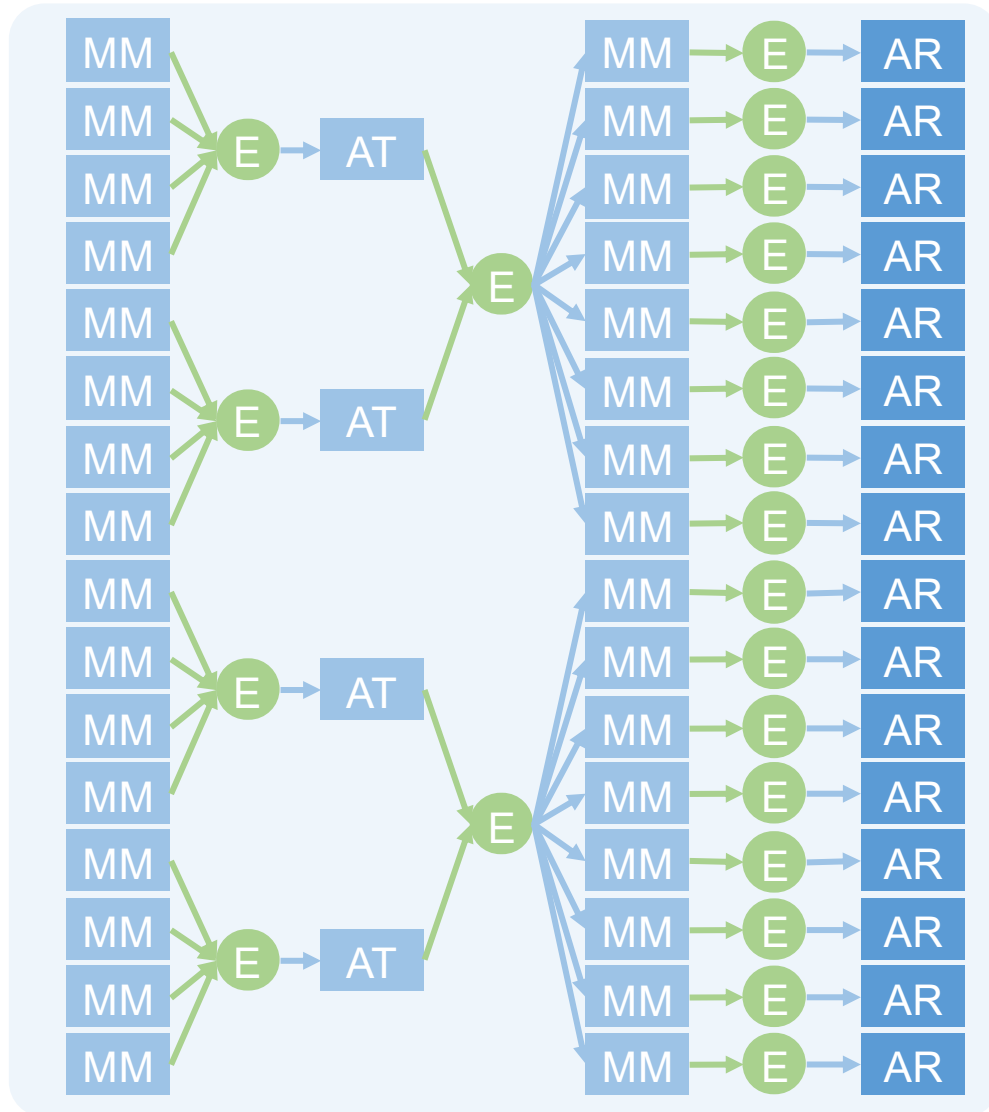
- **Low engineering effort:** a few dozen lines of Python code to mega-kernelize an LLM
- **Better performance:** outperform existing systems by 1.2-6.7x
- **Day-0 support for new models:** do not rely on manual implementation

MPK Overview

Serving config (batching, paging,
speculative decoding, etc)

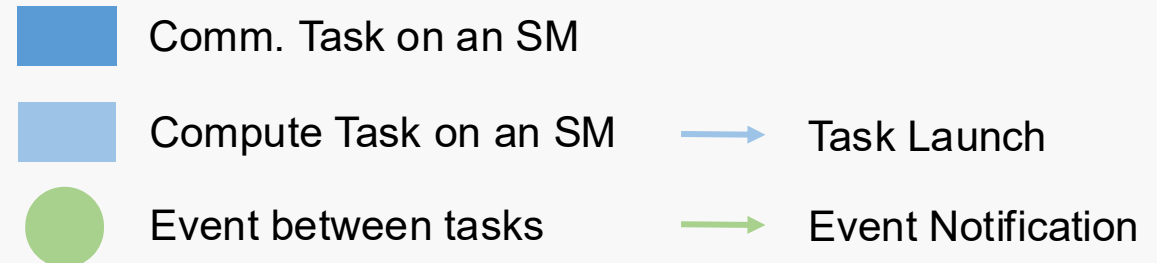


Task Graph



Interleave tasks and events

- **task**: a unit of workload on one SM
- **event**: synchronization among tasks
- **task**→**event**: notify **event** once **task** is done
- **event**→**task**: launch **task** once **event** is triggered

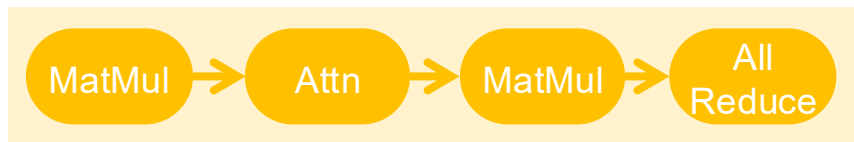


Task Graph vs. CUDA Graph

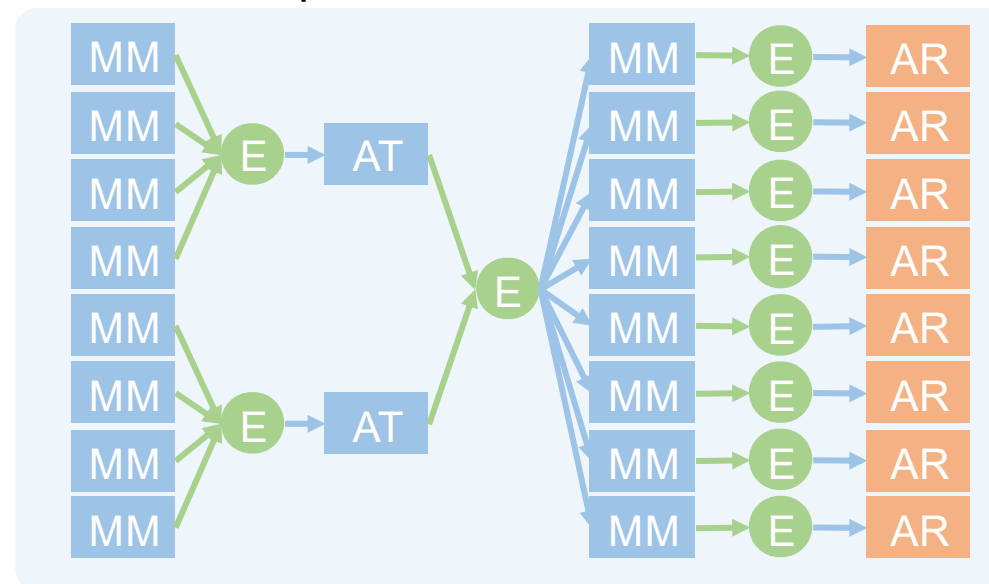
Task graph is a “lower-level” CUDA graph

- Capture sub-kernel dependency
- Static, immutable
- Constructed once and replayed many times

CUDA Graph: nodes are kernels on GPUs

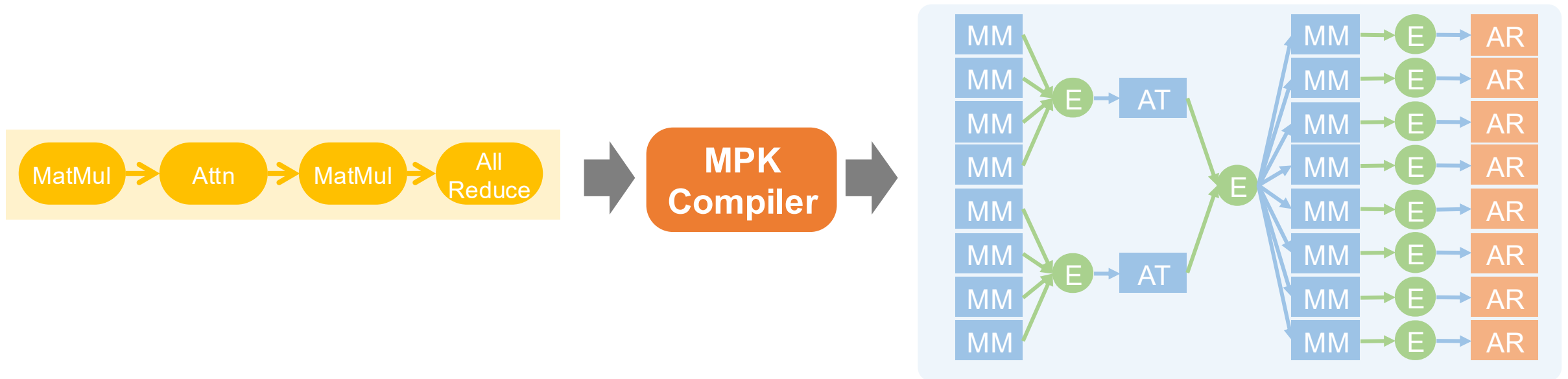


Task Graph: nodes are tasks on SMs

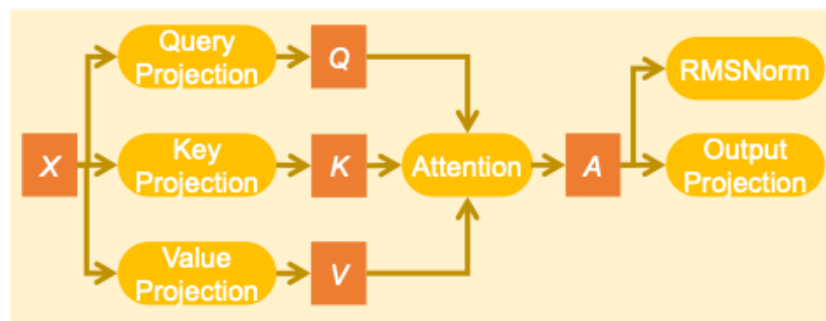


The MPK Compiler

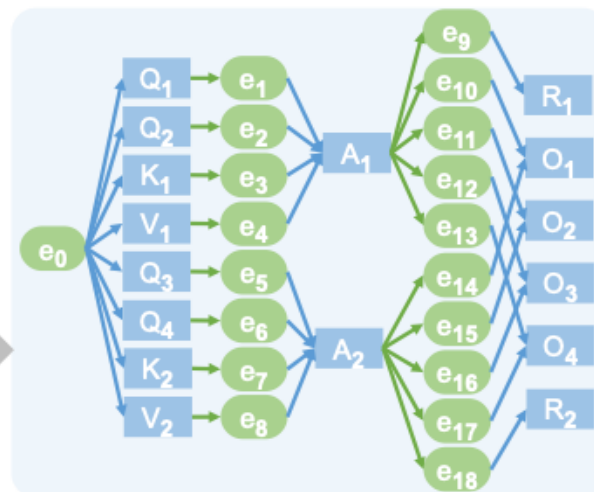
- Optimize operator-to-task decomposition based on available SMs
- Add synchronization events to capture precise task dependencies
- Generate high-performance CUDA implementation for each task



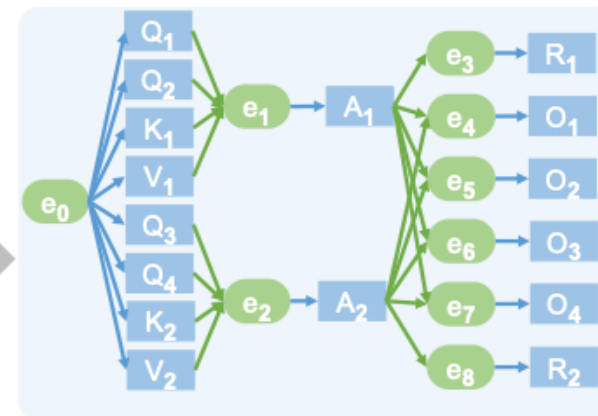
Compiler Workflow



(a) Computation Graph



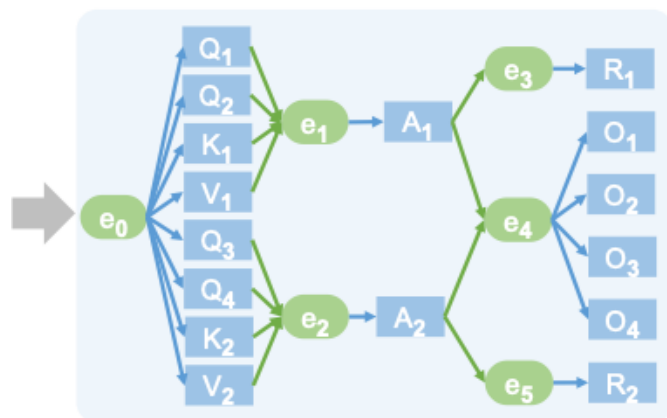
(b) Operator decomposition & dependency analysis



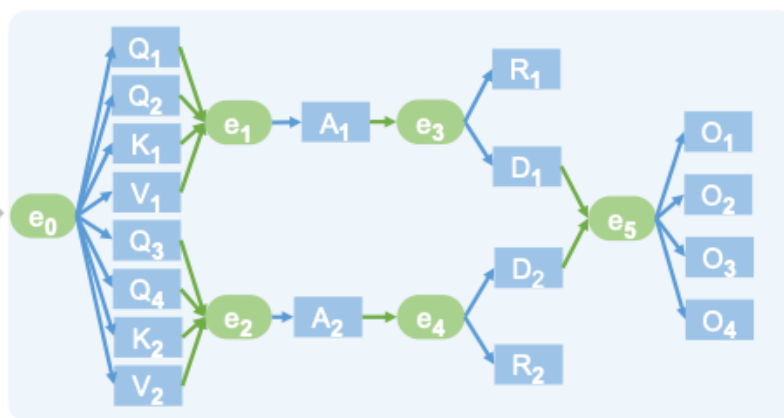
(c) Successor-set fusion

IDs	1	2	3	4	5	6	7	8	9
Tasks	Q ₁	Q ₂	K ₁	V ₁	Q ₃	Q ₄	K ₂	V ₂	A ₁
Depend	e ₀	e ₀	e ₀	e ₀	e ₀	e ₀	e ₀	e ₀	e ₁
Trigger	e ₁	e ₁	e ₁	e ₁	e ₂	e ₂	e ₂	e ₂	e ₃
IDs	10	11	12	13	14	15	16	17	18
Tasks	A ₂	R ₁	D ₁	R ₂	D ₂	O ₁	O ₂	O ₃	O ₄
Depend	e ₂	e ₃	e ₃	e ₄	e ₄	e ₅	e ₅	e ₅	e ₅
Trigger	e ₄	-	e ₅	-	e ₅	-	-	-	-
Events	e ₀	e ₁	e ₂	e ₃	e ₄	e ₅			
Counts		0	4	4	1	2	1		
First task ID	1	9	10	11	13	15			
Last task ID	8	9	10	12	14	18			

(f) *t*Graph linearization



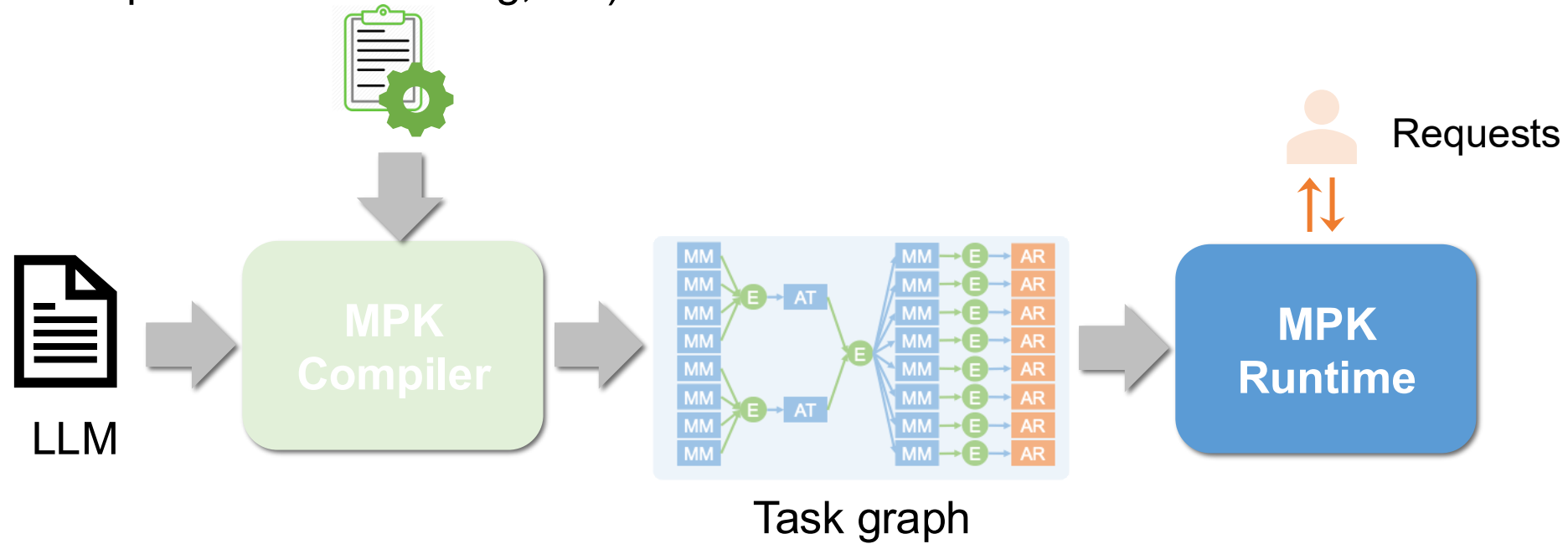
(d) Predecessor-set fusion



(e) *t*Graph normalization

MPK Overview

Serving config (batching, paging,
speculative decoding, etc)



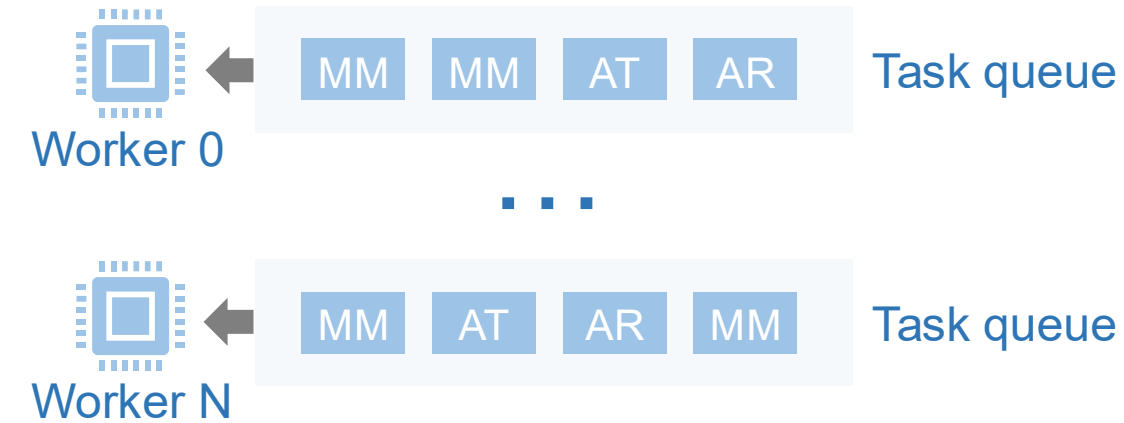
The MPK Runtime

Each scheduler runs on one warp

Each worker runs on one SM



Task-Based Parallel Runtime



Repeatedly

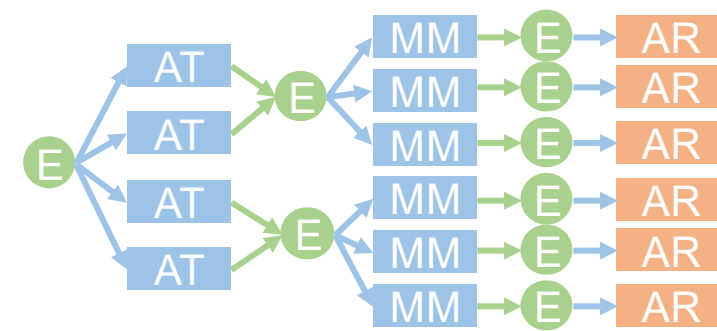
1. Fetch a task from its queue
2. Execute the task
3. Trigger the completion event



Repeatedly

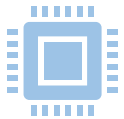
1. Dequeue fully triggered event
2. Launch all tasks depending on the event

Event-Driven Execution

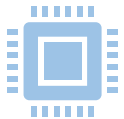


Scheduler
on a warp

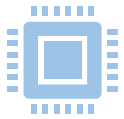
Scheduler
on a warp



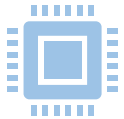
Worker on
an SM



Worker on
an SM

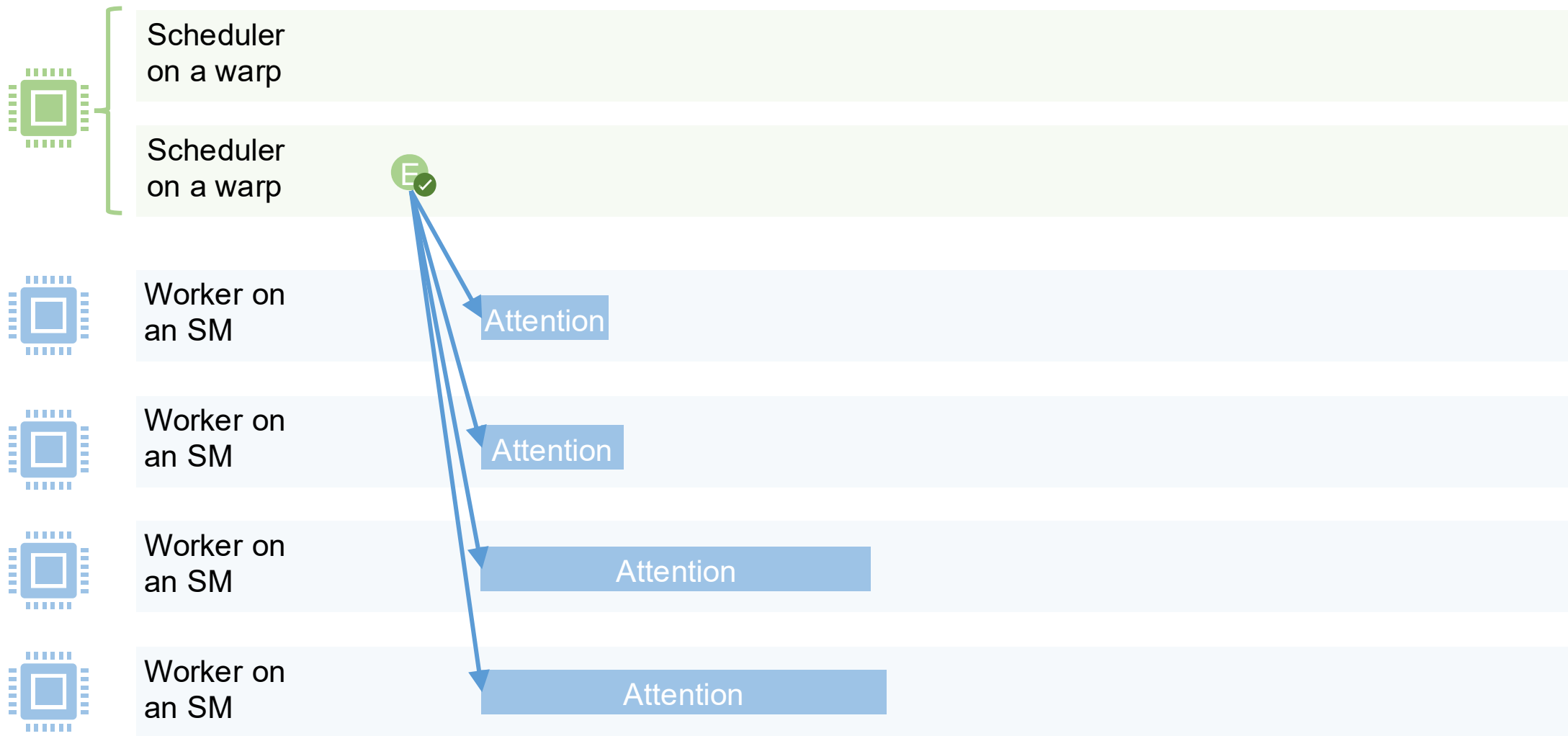
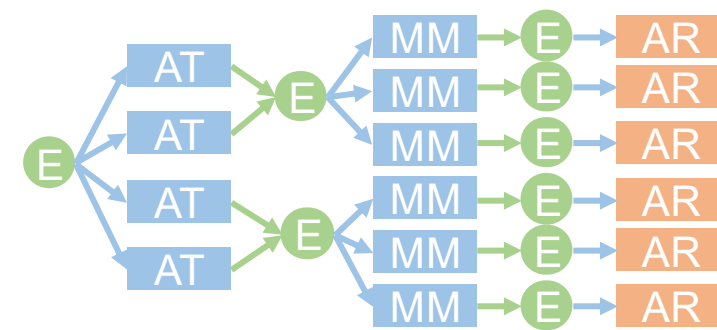


Worker on
an SM

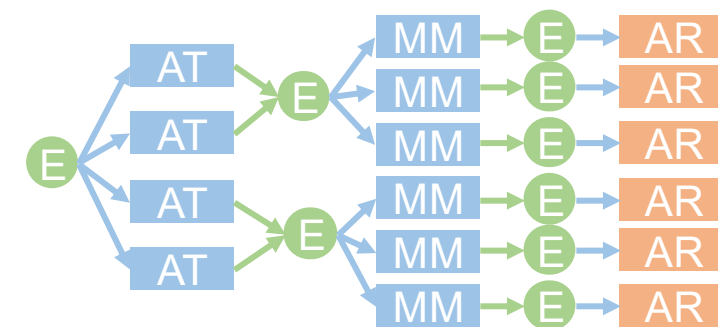
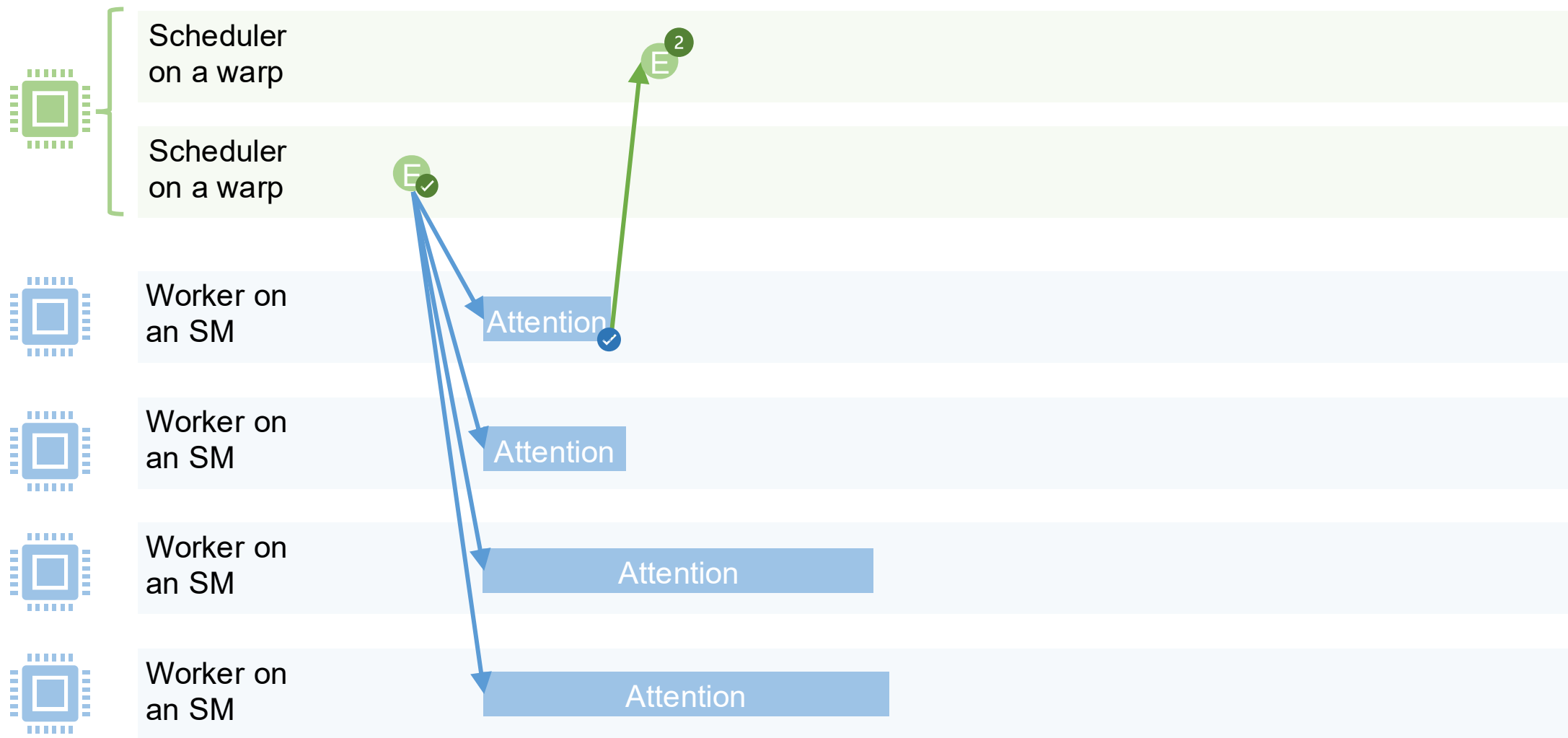


Worker on
an SM

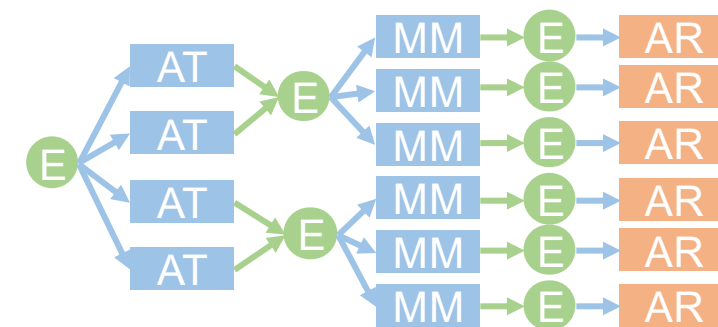
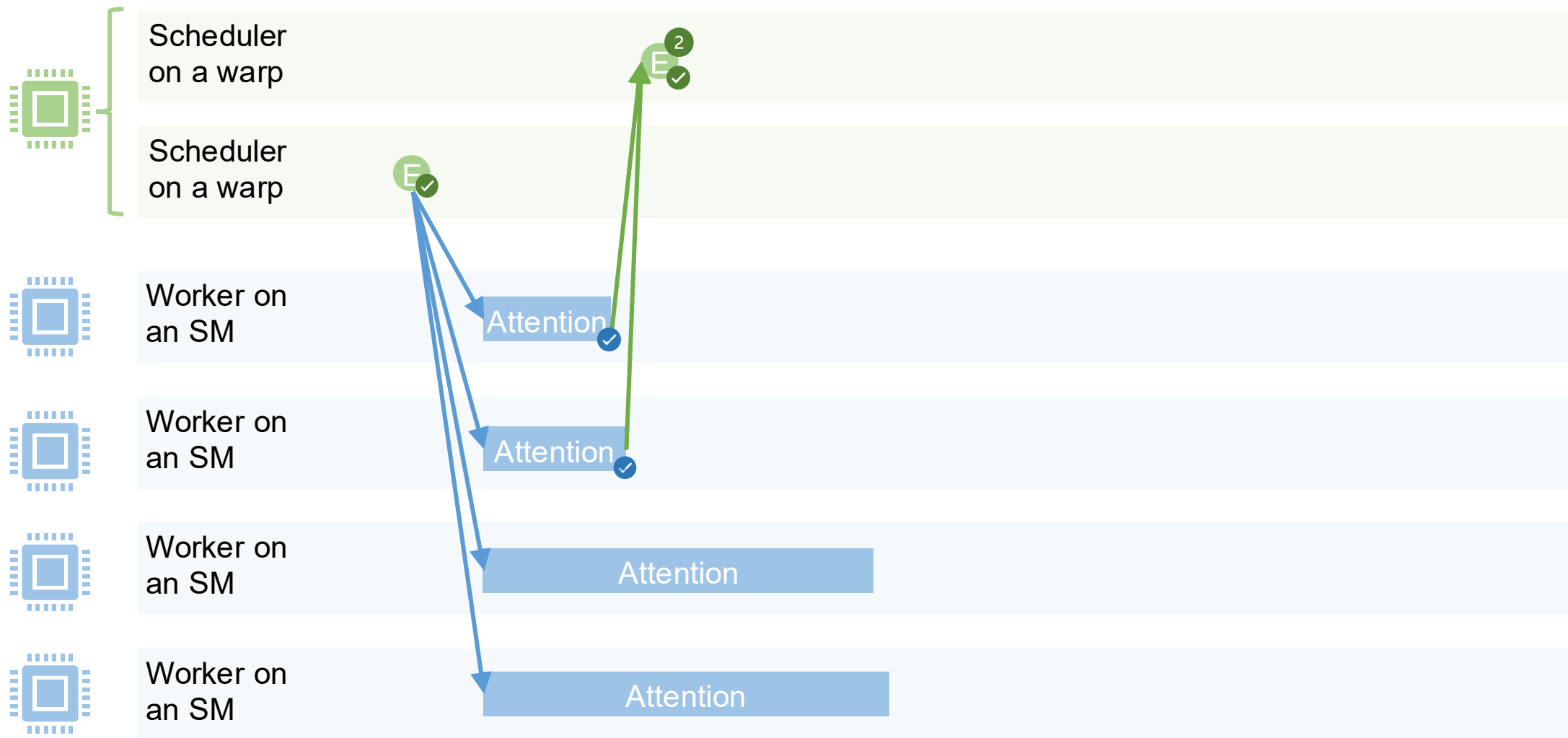
Event-Driven Execution



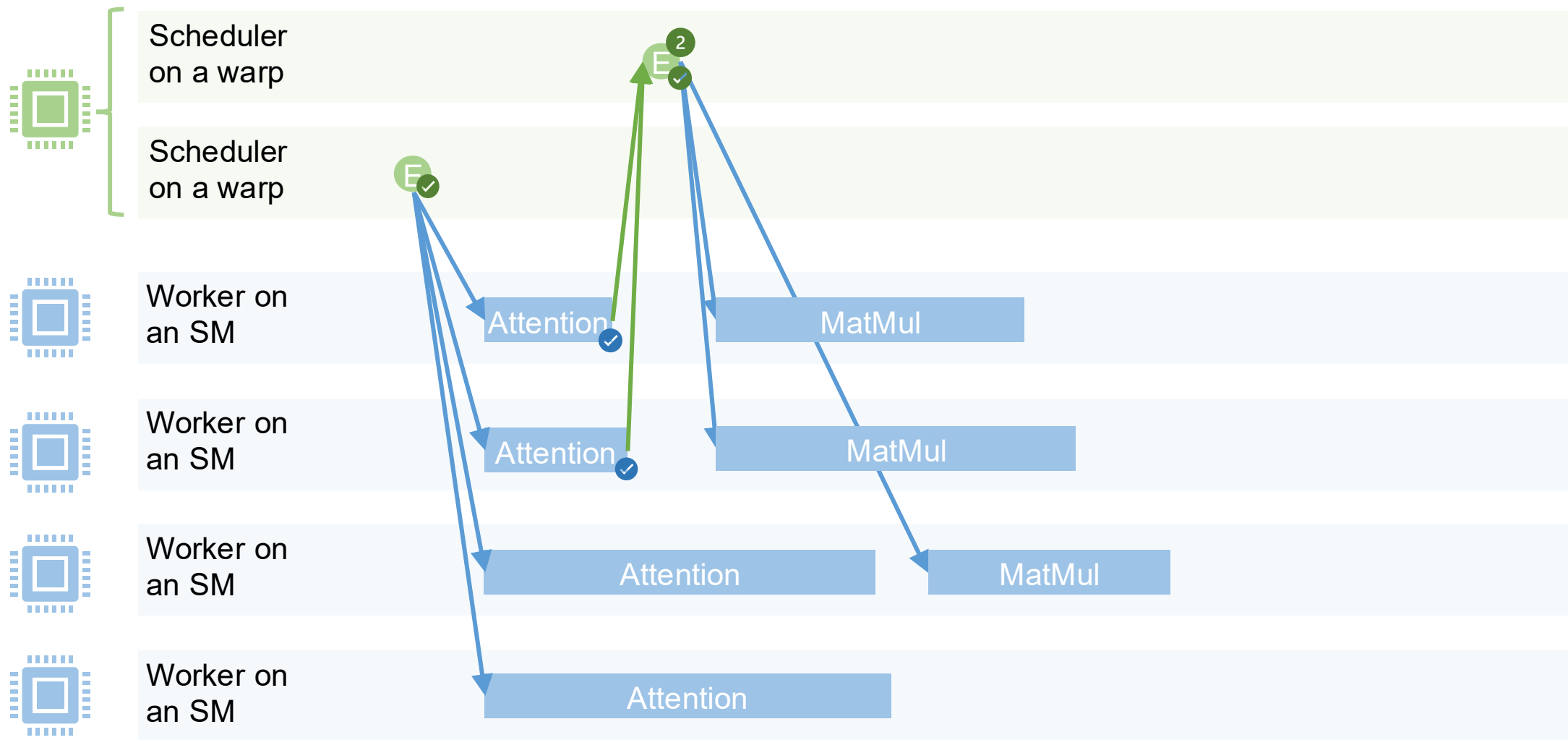
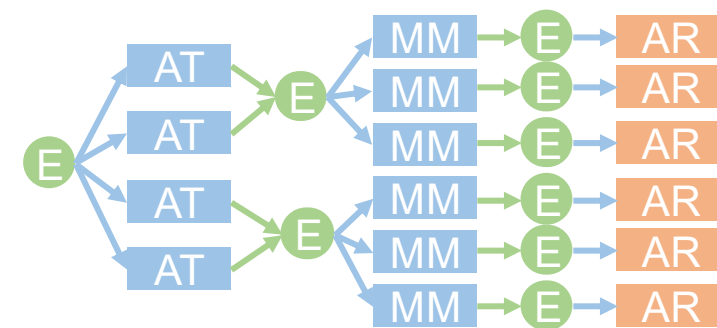
Event-Driven Execution



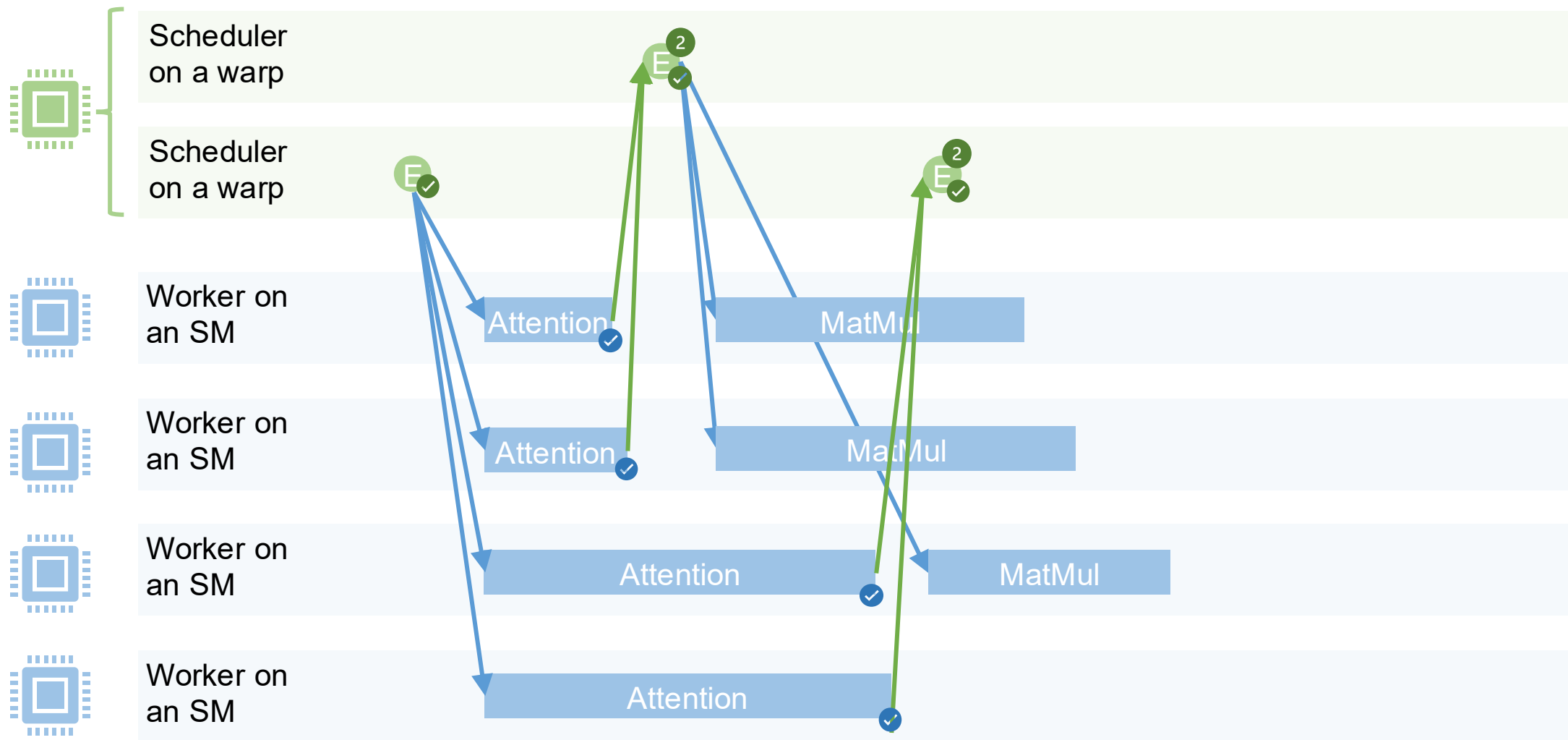
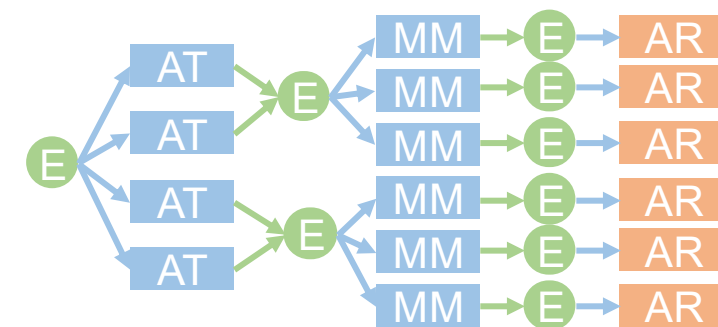
Event-Driven Execution



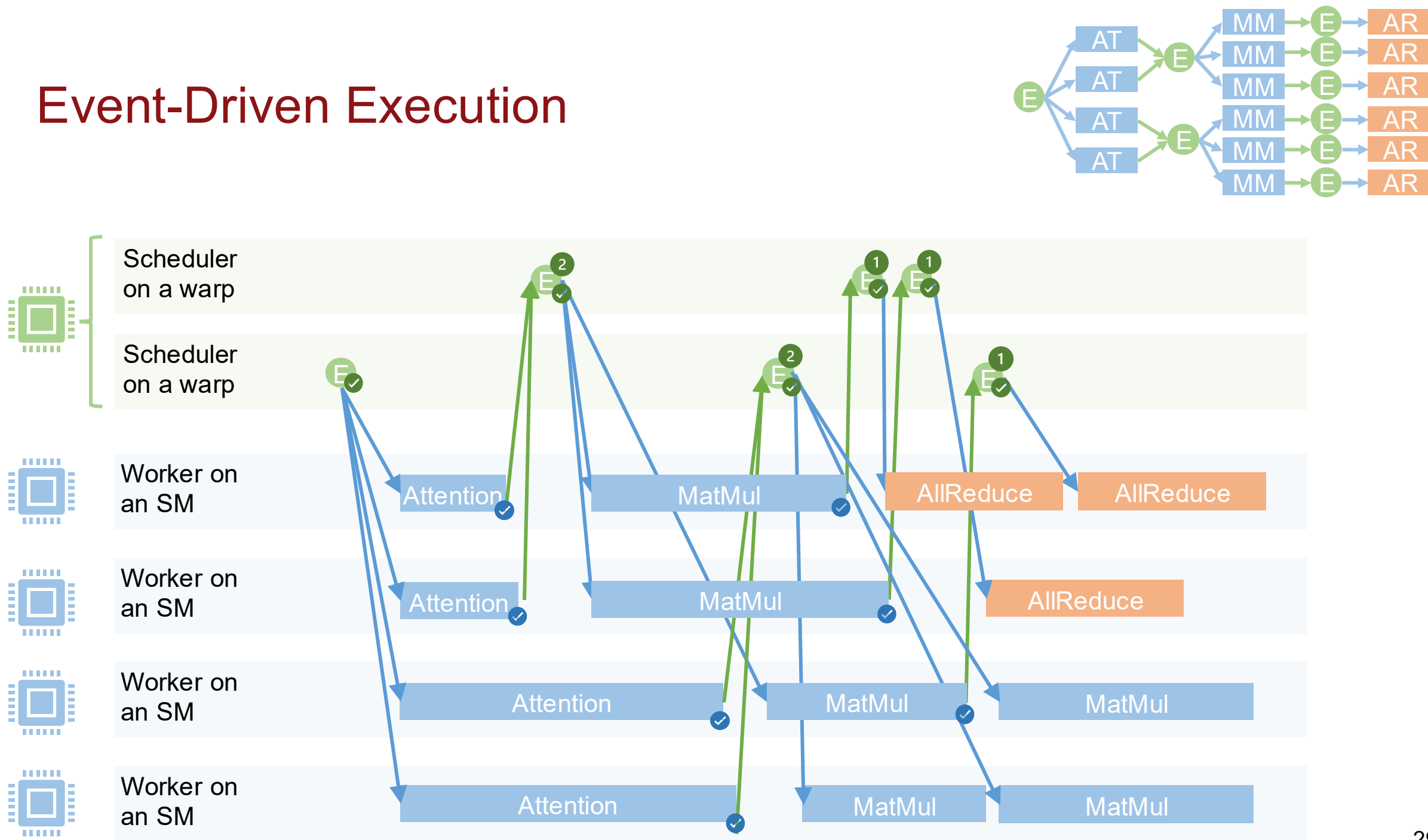
Event-Driven Execution



Event-Driven Execution

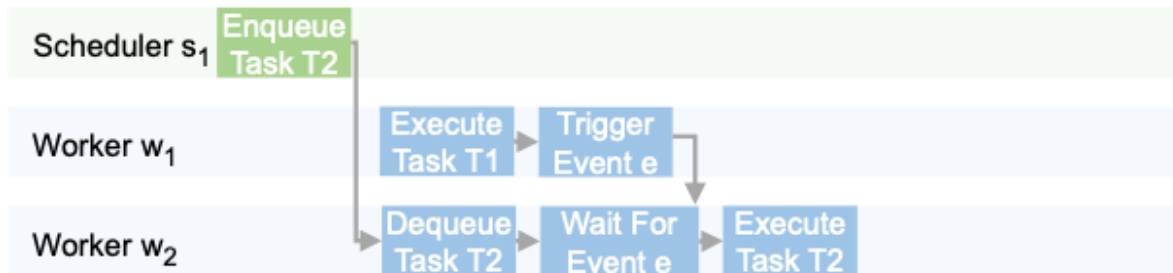


Event-Driven Execution

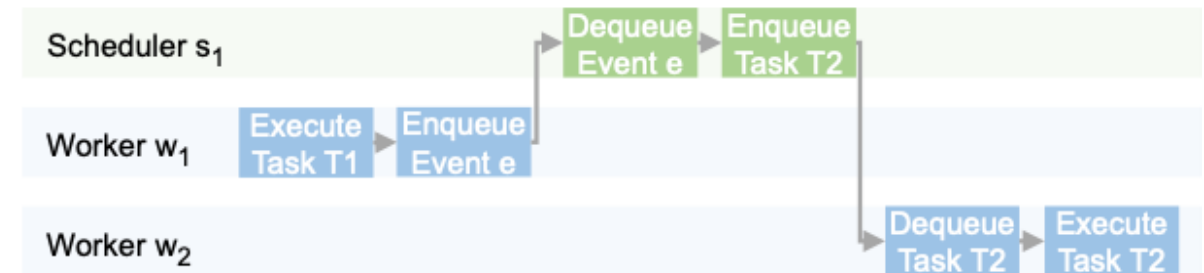


Techniques to Minimize Task Launch Overhead

- **Lightweight workers and schedulers**
 - Task & event queues: circular buffers on device memory
 - Event notification, task enqueue/dequeue: atomic operations
- **Decentralized scheduling**
 - Schedulers assign tasks using only local information
- **Hybrid task launch**
 - Compiler classifies each op as AOT/JIT based on runtime load balance



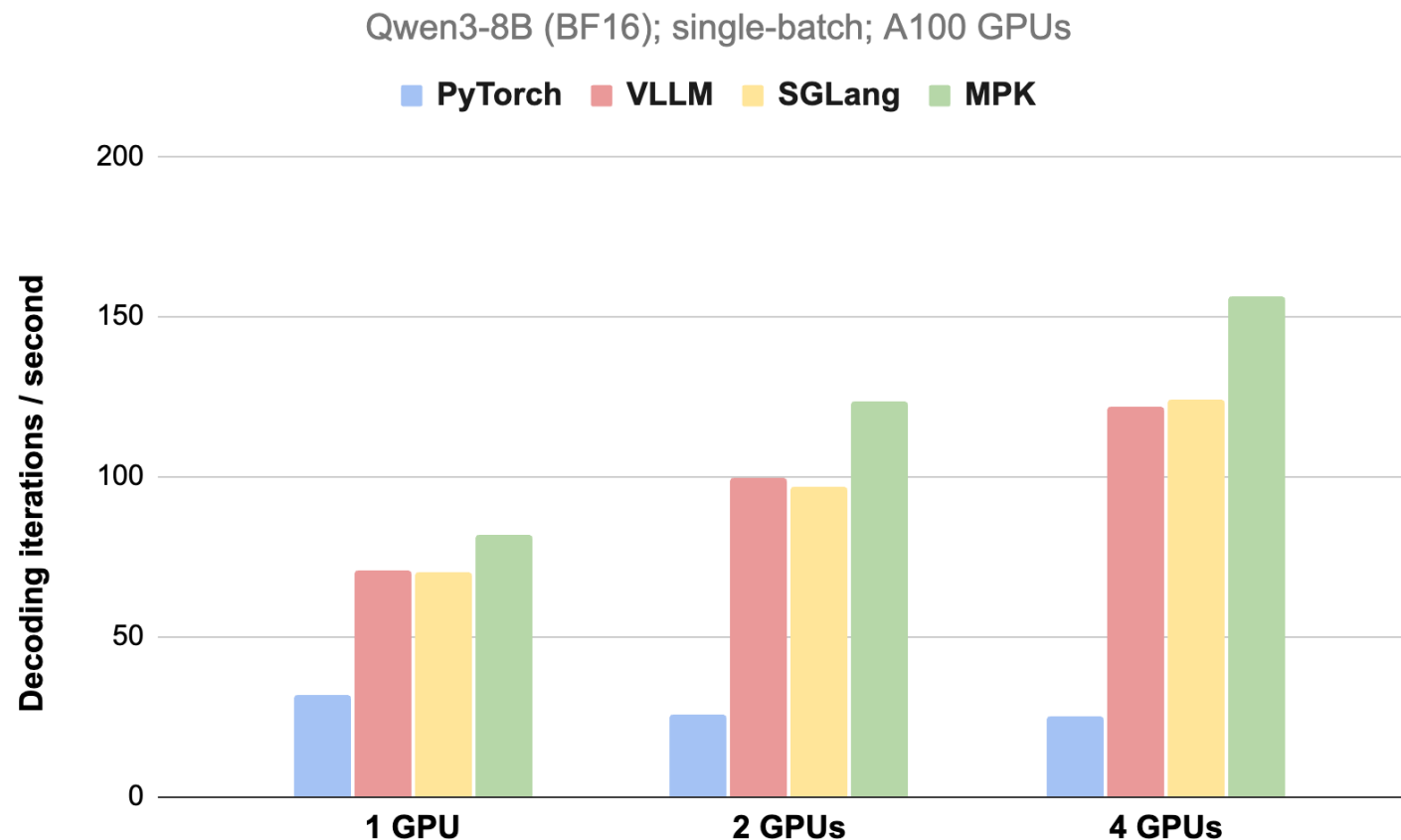
Ahead-of-time: launched before event trigger to reduce latency



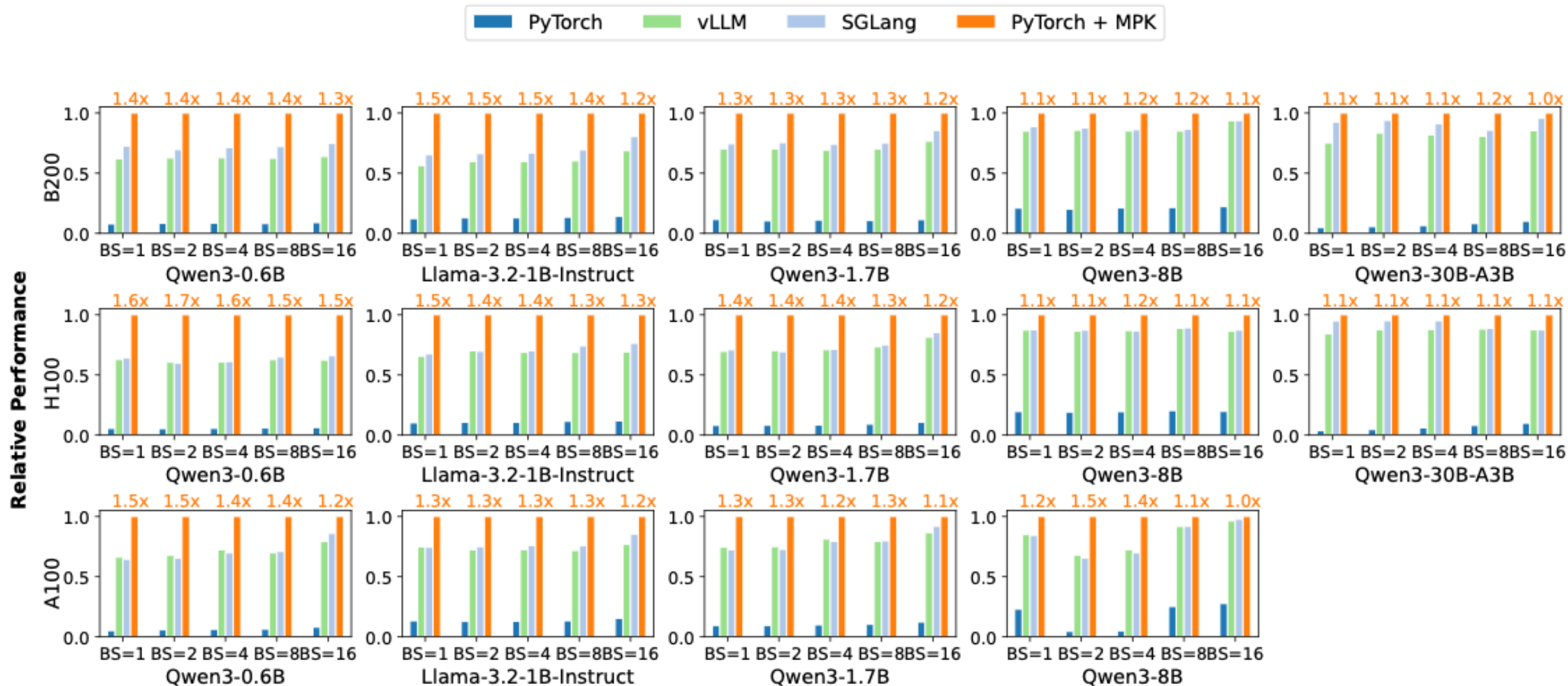
Just-in-time: launched after event trigger to balance load

Pushing LLM Inference Latency Towards HW Limits

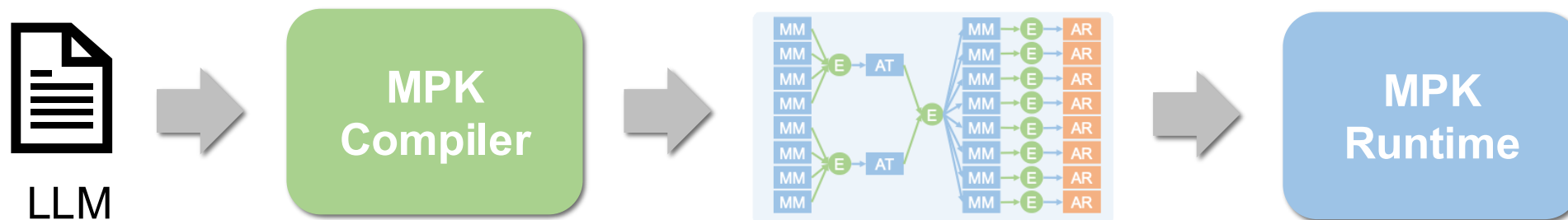
- Reducing Qwen3-8B per-token latency from 14.5ms to 12.5ms
- Approaching theoretical bound of 10ms



Consistently Outperforms Existing Systems Across Models, Batch Sizes, and GPUs



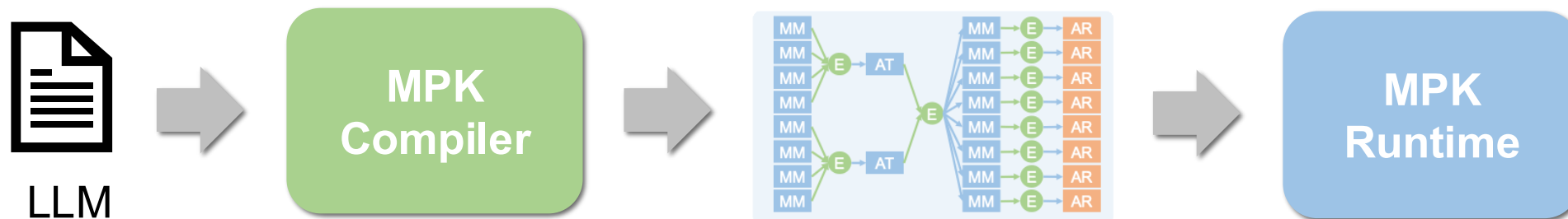
Open Questions



How to decompose layers into tasks?

- Currently rely on heuristics + profiling-based tuning
- Optimized for A100, H100, B200
- Users want portability across diverse GPUs
- Need more automated methods

Open Questions



How to schedule tasks across workers?



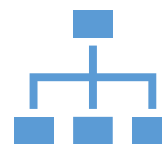
Round-Robin



no
synchronization with
workers



load imbalance



Dynamic Scheduling



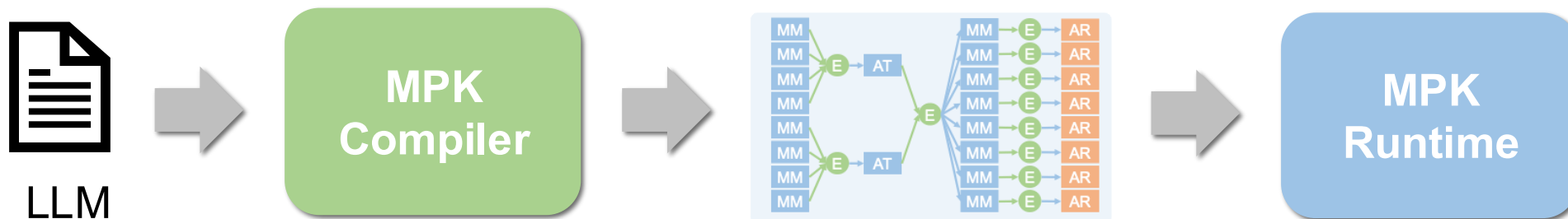
adapt to runtime load



coordination

overhead

MPK: Compiling LLMs into a Mega-Kernel



<https://github.com/mirage-project/mirage/>

Unwatch 29 Fork 184 Starred 2.2k

Hacker News new | past | comments | ask | show | jobs | submit

1. ▲ Compiling LLMs into a MegaKernel: A path to low-latency inference (zhihaojia.medium.com)
134 points by matt_d 4 hours ago | hide | 30 comments
2. ▲ Literate programming tool for any language (github.com/zyedidia)
24 points by LorenDB 1 hour ago | hide | 8 comments
3. ▲ Show HN: I wrote a new BitTorrent tracker in Elixir (github.com/dahrkael)
12 points by dahrkael 1 hour ago | hide | discuss

